

GUIA DE EXPOSICION - PROYECTO SIGOMEI

Arquitectura distribuida con Java RMI, contrato cliente-servidor, ciclo TDD, matriz de trazabilidad, cobertura, demos y preguntas tecnicas.

Tecnologia | Java RMI + JavaFX + JUnit 5 + JaCoCo

Servicio remoto | SIGOMEI publicado por SigomeiServer en el registry RMI

Entidades CRUD | Equipo, Tecnico y Orden

Resultado pruebas | mvn test: 71 pruebas, 0 fallas, 0 errores, 1 omitida manual

1. Apertura sugerida

"Nuestro sistema se llama SIGOMEI y resuelve la gestion de mantenimiento industrial. Lo implementamos como una aplicacion distribuida: el cliente JavaFX invoca un servicio remoto publicado por el servidor mediante Java RMI. La capa de servicio concentra validaciones, reglas de negocio y operaciones CRUD sobre equipos, tecnicos y ordenes."

2. Arquitectura distribuida implementada

- Cliente: SigomeiFxApp. Presenta pantallas, captura datos, filtra informacion y llama metodos remotos del contrato SigomeiRemote.
- Servidor: SigomeiServer crea el registry RMI, instancia SigomeiRemoteImpl y publica el servicio con el nombre SIGOMEI.
- Implementacion remota: SigomeiRemoteImpl extiende UnicastRemoteObject y delega en UsuarioService, EquipoService, TecnicoService y OrdenService.
- Servicios: contienen validaciones, reglas de negocio, CRUD e historial. El cliente no accede directamente a la logica interna.
- Datos compartidos: DTOs y catalogos en api/dto y api/catalogos viajan entre cliente y servidor.

Lectura rapida

El cliente solo conoce el contrato. El servidor conserva la autoridad de negocio. Si una regla falla, el servidor lanza una excepcion de dominio y el cliente la muestra al usuario.

3. Justificacion tecnica de RMI

- Encaja con el proyecto porque cliente y servidor estan hechos en Java.
- Permite invocar metodos remotos tipados casi como metodos locales.
- Evita construir un protocolo manual con sockets: codigos de operacion, parseo, serializacion y manejo de errores.
- Transporta DTOs y excepciones de dominio, lo que hace mas claro el contrato.
- Su trade-off es que acopla a Java; aun asi, para este proyecto academico Java-Java reduce complejidad y favorece la claridad.

4. Contrato de comunicacion

El contrato esta en SigomeiRemote, que extiende Remote. Cada metodo declara RemoteException y, cuando aplica, excepciones de dominio: ValidacionException, ReglaNegocioException, RegistroNoEncontradoException y AutenticacionException.

Modulo | Operaciones principales | Errores controlados

Sesion | iniciarSesion, cerrarSesion | Credenciales invalidas o usuario inactivo

Equipo | registrar, consultar, filtrar, actualizar, cambiar estado, eliminar | Campos obligatorios, serie duplicada, equipo relacionado a orden

Tecnico | registrar, consultar, filtrar, actualizar, cambiar estatus, eliminar | RFC/correo duplicado, tecnico con ordenes activas

Orden | registrar, consultar, filtrar, actualizar, cambiar estado, historial, eliminar | Tecnico incompatible, duplicidad, fechas invalidas, transicion no permitida

5. Ciclo TDD aplicado

Etapas | Aplicacion en SIGOMEI

Plan | Se identificaron entidades, RF, reglas RN01-RN10 y casos CP01-CP35.

Diseño | Se escribieron pruebas positivas y negativas en ReglasNegocioServiceTest y ValidacionesServiceTest.

ROJO | La prueba fallaba cuando la regla aun no existia o no lanzaba la excepcion esperada.

VERDE | Se implemento la logica minima en servicios para pasar la prueba.

Ejecucion | Se corrio mvn test con JUnit 5 y Surefire.

Reporte | JaCoCo genero cobertura de la capa com.sigomei.servidor.service.

Frase clave: TDD no fue probar al final; las pruebas definieron que se acepta, que se rechaza y que excepcion debe regresar el servicio.

6. Matriz de trazabilidad RF/RN - Caso - Estado

RF/RN | Caso | Estado

RF-01 Iniciar sesion | CP01 + usuario_cubreAutenticacionCorrectaElIncorrecta | Aprobado

RF-02 CRUD Equipo | CP02-CP04 + CrudServiceTest.crudEquipo | Aprobado

RF-03 CRUD Tecnico | CP05-CP09 + CrudServiceTest.crudTecnico | Aprobado

RF-04 CRUD Orden | CP10-CP16 + CrudServiceTest.crudOrden | Aprobado

RN-01 Tecnico compatible | rn01 positivo/negativo | Aprobado

RN-02 Sin orden activa duplicada | rn02_negativo_ordenDuplicada + CP23 | Aprobado

RN-03 No eliminar/inactivar con ordenes | rn03 + CP30 + CP31 | Aprobado

RN-04 Tecnico activo | rn04_negativo_tecnicoInactivo + CP24 | Aprobado

RN-05 Fechas cronologicas | rn05 + CP26 | Aprobado

RN-06 Finalizacion completa | rn06 + CP28 | Aprobado

RN-07 Criticidad alta | rn07 + CP25 | Aprobado

RN-08 Transiciones de estado | rn08 + CP27 | Aprobado

CP-35 Desconexion durante operacion | Prueba deshabilitada | Manual

7. Reporte de cobertura

Metrica | Resultado | Interpretacion

Instrucciones | 1839/1858 = 98.98% | La mayoría de la logica de servicio fue ejecutada

Ramas | 296/320 = 92.50% | Se probaron condiciones positivas y negativas

Lineas | 343/346 = 99.13% | Casi todas las lineas de servicio fueron recorridas

Metodos | 51/51 = 100% | Todos los metodos de servicio incluidos fueron ejercitados

Maven | 71 pruebas, 0 fallas, 0 errores, 1 omitida | La omitida es manual por desconexion del servidor

8. Demostracion en vivo recomendada

CRUD remoto de 3 entidades

- Levantar SigomeiServer desde el IDE o comando configurado.
- Abrir SigomeiFxApp y entrar con admin / admin123.
- Equipo: registrar, filtrar y actualizar ubicacion o estado.
- Tecnico: registrar, editar telefono y eliminar si no tiene ordenes.
- Orden: registrar con equipo/tecnico compatibles, actualizar descripcion y cambiar de PROGRAMADA a EN_EJECUCION.

Tres casos obligatorios

Caso | Que mostrar | Mensaje

Unitaria con memoria | mvn -Dtest=ReglasNegocioServiceTest test | InMemorySigomeiStore aísla la capa de servicio sin depender de MySQL

Negativo GUI | Dejar fecha vacía o mal formada en Equipo | La GUI captura la excepción y muestra diálogo de error

Regla de negocio | Orden con técnico incompatible o duplicada | El servidor rechaza con ReglaNegocioException

Tres reglas aceptado/rechazado

Regla | Aceptado | Rechazado

RN-01 | Equipo eléctrico + técnico eléctrico | Equipo eléctrico + técnico mecánico

RN-02 | Mismo equipo en fecha distinta | Mismo equipo, misma fecha, ambas activas

RN-06 | Finalizada con fecha de cierre y costo real | Finalizar sin fecha o sin costo

9. Analisis critico y recomendaciones

- Defecto/riesgo: CP-35 queda manual porque depende de apagar el servidor durante una operación.
- Riesgo: la persistencia de prueba usa memoria; conviene ampliar pruebas con MySQL para integración real.
- Riesgo: la GUI captura excepciones generales; sería mejor diferenciar validación, regla de negocio y conexión.
- Valor de TDD: documento reglas como comportamiento verificable, evita regresiones y facilita la matriz de trazabilidad.
- Recomendaciones: automatizar desconexión controlada, agregar validaciones visuales previas, separar repositorios por interfaces y mantener JaCoCo en cada entrega.

10. Preguntas probables con respuestas

P: Por qué RMI y no sockets?

R: Porque ambos extremos están en Java. RMI da invocación remota tipada, serialización de DTOs y excepciones sin protocolo manual.

P: ¿Cuál es el contrato?

R: SigomeiRemote. Define métodos, DTOs y excepciones que puede recibir el cliente.

P: ¿Qué viaja por la red?

R: DTOs y parámetros simples: IDs, fechas, catálogos, textos y costos; no viajan servicios ni repositorios.

P: ¿Dónde están las reglas?

R: En la capa service: OrdenService, EquipoService y TecnicoService.

P: ¿Cómo aplicaron TDD?

R: Primero pruebas positivas/negativas, luego implementación mínima para pasar de rojo a verde, después suite completa y JaCoCo.

P: ¿Qué significa rojo/verde?

R: Rojo: la prueba falla porque la regla no existe. Verde: la lógica se implementa y la prueba pasa.

P: ¿Cómo saben que el CRUD remoto funciona?

R: El contrato expone CRUD de Equipo, Técnico y Orden; CrudServiceTest lo verifica y la demo JavaFX lo invoca vía RMI.

P: ¿Qué pasa si el servidor no está disponible?

R: La llamada RMI falla con excepción remota/conexión. CP32 cubre servidor no disponible y CP35 queda manual para desconexión durante operación.

P: ¿Por qué DTOs?

R: Separan datos de transporte de la logica interna y evitan exponer implementaciones del servidor.

P: Que regla es mas importante?

R: RN-02 evita duplicidad de ordenes activas y RN-06 obliga cierre completo con fecha y costo real.

P: La memoria reemplaza MySQL?

R: No. Sirve para pruebas unitarias rapidas y repetibles; MySQL corresponde a persistencia real o integracion.

P: Que cubre JaCoCo?

R: La capa com.sigomei.servidor.service, donde viven reglas y validaciones principales.

P: Limitacion de RMI?

R: Acopla a Java y es menos interoperable que HTTP/REST, pero simplifica un proyecto Java-Java.

P: Como repartir participacion?

R: Arquitectura, contrato RMI, TDD/cobertura, demo CRUD, reglas/analisis; todos deben responder al menos una pregunta.

11. Reparto sugerido

Integrante | Parte | Tiempo

1 | Problema, arquitectura cliente-servidor y RMI | 2 min

2 | Contrato SigomeiRemote, DTOs y excepciones | 2 min

3 | Ciclo TDD, pruebas unitarias y cobertura | 3 min

4 | Demo CRUD remoto de Equipo, Tecnico y Orden | 4 min

5 | Demo de reglas, matriz, analisis y recomendaciones | 3 min

Cierre: SIGOMEI separa cliente, contrato remoto y servicios; RMI simplifica la comunicacion Java-Java; TDD permitio convertir reglas de negocio en pruebas verificables.